

02 — Writing Verbs

A verb is a Python file. When a player types a command that resolves to a verb, the engine builds a namespace, preprocesses the source, and executes it. This document is the reference for what you can rely on inside that file.

- [The shape of a verb](#)
- [The verb namespace](#)
- [The source preprocessor](#)
- [Matching objects](#)
- [Verb types: customizing the parser](#)
- [Messaging and emit substitution](#)
- [Calling other verbs](#)
- [Creating and changing objects](#)
- [Timing: pause, delay, and fork](#)
- [Interactive verbs with `yield`](#)
- [Permission-checked attribute access](#)
- [Conventions and idioms](#)

The shape of a verb

A verb file is bare Python — no function definition, no imports, no boilerplate. It runs top to bottom and may `return` at any point (the engine wraps the body in a function so top-level `return` is legal). Standard library `import` works when you need it, but most verbs need nothing beyond the injected namespace.

```
"""
Optional docstring. The first lines often serve as the in-game help text.

Usage: <verb> <args>
"""

if not args:
    pobj.msg("Do what?")
    return

# ... logic ...
```

The leading triple-quoted docstring is conventional: it documents usage and is where in-game help comes from.

The verb namespace

The namespace is assembled by `build_verb_namespace()` in `moo/verb_namespace.py`. Everything below is available as a bare name inside any verb.

Core context

Name	What it is
<code>pobj</code> / <code>player</code>	The acting player object (the two names are the same object).
<code>this</code>	The object the verb is defined on . Use it for the verb's own properties and helpers.
<code>caller</code>	The object that invoked this verb via <code>call_verb</code> (<code>None</code> for a top-level command).
<code>db</code>	The database singleton. <code>db.get_object(N)</code> fetches by number.
<code>location</code>	The player's current location (or <code>None</code>).
<code>verb</code>	The verb name as matched, e.g. <code>'look'</code> .
<code>args</code>	The argument string, stripped.
<code>argstr</code>	The raw, unstripped argument string as typed.

`pobj` vs `this` is the distinction that trips people up first: in `moo/verbs/17/jump.py`, `pobj` is the player jumping and `this` is the `ICRoom` the verb lives on. In a verb defined on a sword, `this` is the sword and `pobj` is whoever swung it.

Parsed command parts

Every one of these is a string (or a list of strings). None of them is a resolved object. `dobj` and `dobjstr` are the *same string*, as are `iobj` and `iobjstr` — the pair exists for familiarity, not because one is matched. Turning text into an object is always your verb's job; see [Matching objects](#).

Name	What it is
<code>dobj / dobjstr</code>	Direct-object text . Identical values.
<code>iobj / iobjstr</code>	Indirect-object text . Identical values.
<code>prep / preplist</code>	The preposition (<code>'in'</code> , <code>'on'</code> , <code>'from'</code> , ...) and its tokens.
<code>dobjlist / iobjlist</code>	The dobj/iobj strings split into tokens.
<code>dobj2 / prep2</code>	Secondary object text / preposition for multi-preposition commands.
<code>lhs / rhs</code>	Left/right of the preposition, for assignment-style verbs (<code>@desc x = y</code>).
<code>arglist</code>	<code>argstr</code> split on whitespace.
<code>switches</code>	Slash switches: <code>look/brief</code> → <code>['brief']</code> .
<code>match</code>	The preposition regex match object, or <code>None</code> .

For `put gem in box` you get `dobj == dobjstr == 'gem'` , `prep == 'in'` , and `iobj == iobjstr == 'box'` . To act on the gem you must match it yourself:

```
gem = pmatch(dobj, pobj, list(pobj.contents))
```

The command parser (`moo/parser.py`) *does* attempt its own object resolution and records object numbers on the `ParseResult` , but those numbers are discarded: the verb type's string parse overwrites them when the namespace is built. Its `ArgSpec` enum (`NONE / ANY / THIS`) is vestigial for the same reason — `VerbDef` has no `dobj_spec / iobj_spec` fields to declare, so the permissive defaults always apply.

Python builtins

A curated set is injected by name (`SAFE_PYTHON_BUILTINS` in `moo/verb_namespace.py`): `len` , `str` , `int` , `float` , `bool` , `list` , `dict` , `set` , `tuple` , `type` , `range` , `enumerate` , `zip` , `map` , `filter` , `reversed` , `sorted` , `sum` , `min` , `max` , `abs` , `round` , `all` , `any` , `isinstance` , `print` — plus the permission-checking `getattr / setattr` wrappers and a plain `hasattr` .

This is a convenience layer, not a security boundary. The namespace is a plain dict with no `__builtins__` key, and CPython injects the real builtins module into any such dict at `exec()` time. So `import` , `open` , `eval` , `exec` , and `__import__` all work inside verb code, and `import builtins; builtinsgetattr(...)` sidesteps the permission wrappers. The engine relies on this: `@adverb` itself does `import re` and `import os` to write verb stubs to

disk. Treat verb code as **trusted** code — the gm3 tier that can write verbs is effectively root on the server, which is why the permission ladder matters more than the builtin list.

The MOO builtin library

Injected from `moo/builtins.py`. Grouped by purpose:

Objects: `create(parent, owner)`, `recycle(obj)`, `valid(obj)`, `get_object(objnum)`, `move(obj, dest)`, `chparent(obj, new_parent)`, `max_object()`.

Properties: `add_property(obj, name, value=None, perms='rc')`, `delete_property(obj, name)`, `properties(obj)`.

Verbs: `add_verb(obj, names, perms='rx', ...)`, `delete_verb(obj, name)`, `verbs(obj)`.

Calling & scheduling: `call_verb(obj, verb_name, ..., **kwargs)`, `pause(seconds)`, `delay(seconds, code, context)`, `fork(seconds, code, context)` — see [Timing](#).

Messaging: `obj.msg(message, ...)` and `room.msg_room(message, ...)` are the verbs you call from verb code (defined on #1, so every object has them); `broadcast(message, ...)` reaches everyone connected. See [Messaging](#).

Matching: `pmatch`, `bmatch`, `match`, `match_all`, `omatch`, `smatch`, plus helpers `strip_articles`, `parse_ordinal`, `name_match`, `adj_match`, `prep_match`, `split_on_prep`.

Search: `search(query, ...)` / `find(query, ...)`.

Admin / lifecycle: `auth_level(obj)`, `sync_auth_flags(obj)`, `puppet(target)`, `force(player, command)`, `fire_hook(...)`, and the ticker functions `ticker_add(interval, verb_name, obj, idstring='')` / `ticker_remove(...)` / `ticker_remove_all(obj)` — hooks and the ticker are covered in Engine Systems.

Helper modules: `su` (string utilities — emit/pronoun substitution), `ou` (object utilities — `make_room`, `make_object`, ...), `_effects` (the effects manager — note there is no bare `eu`; use `$eu`), and `globals` (the `moo.globals` module of shared constants — this shadows Python's `globals()` builtin).

The source preprocessor

Before compilation, verb source is rewritten by `preprocess_verb_code()` (`moo/verbs.py`). Two transformations matter:

Reference rewriting

A single master regex scans the source and rewrites object and system references **only in bare code** — strings and comments are matched first and left untouched.

You write	Becomes	Notes
<code>#42</code>	<code>db.get_object(42)</code>	An object literal.
<code>\$chair</code>	<code>db.get_object(0).chair</code>	A property on the system object (#0) — see named constants below.
<code>\$su</code>	<code>su</code>	The one special case: <code>su</code> is in <code>__PYTHON_CONSTANTS</code> , so it maps to the injected namespace name rather than a <code>#0</code> property.
<code>"text with #5"</code>	unchanged	Inside a string.
<code># a comment about #1</code>	unchanged	A real comment (<code>#</code> not followed by a digit).

The rewrite also reaches **inside f-string expressions**, so `f"You see {#5.name}"` correctly becomes `f"You see {db.get_object(5).name}"`. The distinction between “comment” and “object ref” is precisely “`#` followed by a digit is an object; otherwise it’s a comment,” which is why `#42` and `# note` coexist safely.

Named object constants

The `$` prefix gives well-known objects readable names. Every `$name` (except the special-cased `$su`) rewrites to `db.get_object(0).name` — a property read on the **system object #0**. Because those properties store object numbers, `$name` resolves to a well-known object without hardcoding its number:

```
# Instead of memorizing that the chair prototype is #24:
seat = create(parent=$chair)           # → create(parent=db.get_object(0).chair)
if target.parent == $chest:             # readable, refactor-proof
    ...
$eu.trigger(pobj, 'poison', 5, 3)       # $eu → db.get_object(0).eu (the effects object,
                                         #33)
```

The constants defined on #0 in the shipped database are the prototypes `$bed`, `$chair`, `$chest`, `$furniture`, `$item`, `$obj` and `$table`; `$config` and `$globals`; and the utility objects, each under a long name and a short one — `$string_utils` / `$su`, `$list_utils` / `$lu`, `$match_utils` / `$mu`, `$object_utils` / `$obj_utils` / `$ou`, `$command_utils` / `$cu`, `$code_utils` / `$cdu`, `$perm_utils` / `$pu`, `$effects_utils` / `$eu`, `$help_utils` / `$hu`. A `$name` with no matching property on #0 does not error — it reads as the falsy `_null_attr` sentinel, so a typo fails silently. Check the live list with `+props #0`.

This is the same indirection LambdaMOO’s `$foo` corewords provide: a layer of named aliases over raw object numbers, so verb code reads in terms of *roles* (“the chair prototype”, “the effects utility”) rather than magic numbers, and a world can renumber its prototypes by re-pointing `#0`’s properties.

Adding one: set a property on #0 to the target object number — `@adprop #0.weapon = 200` (or from verb code, `db.get_object(0).weapon = 200`). After that, `$weapon` resolves to `#200` everywhere. The authoritative list of constants is simply whatever properties exist on #0; inspect it live with `+props #0`. Use short, lowercase names that name the role, and avoid Python keywords or builtin names.

Note: `su` and `ou` (string and object utilities) are injected directly into the namespace, so you write `su.wrap(...)` / `ou.make_room(...)` with no `$`. The effects manager is **not**: there is no bare `eu` name in the namespace. Reach it as `$eu` (which rewrites to `db.get_object(0).eu`, i.e. #53), or via the injected `_effects` module object.

Function wrapping

The body is wrapped so `return` works at top level:

```
# what you write
pobj.msg("Hello")
result = "done"

# what runs
def _verb():
    pobj.msg("Hello")
    result = "done"
result = _verb()
```

The wrapper's return value is captured as `namespace['result']` and becomes the value `call_verb` hands back to a caller. Compiled code is cached, so the preprocessing cost is paid once per verb version, not per invocation.

Matching objects

Turning "2nd blue sword" into the object the player means is the job of the matchers in `moo/match_utils.py`. Both major matchers resolve in the same order — possessives (`my sword` restricts to inventory), keywords (`me`, `here`), dbrefs (`#3`, `$utils`), then name matching against a candidate list.

Function	Returns	Use when
<code>pmatch(inp, pobj, candidates)</code>	The single best match, or <code>None</code> .	Player verbs. Supports <code>me</code> , <code>here</code> , <code>my <X></code> — but deliberately not <code>#N</code> or <code>\$name</code> , so players can't reach arbitrary objects.
<code>bmatch(inp, pobj, candidates, db=None)</code>	The single best match, or <code>None</code> .	Staff verbs. Same as <code>pmatch</code> plus <code>#N</code> / <code>\$name</code> dbref support.
<code>match(inp, candidates)</code>	The single best name match, or <code>None</code> (no keywords/refs).	Low-level matching against an explicit list.
<code>match_all(inp, candidates)</code>	A list of every name match.	When you genuinely want all matches.
<code>omatch(inp, pobj, db=None)</code>	Keyword/ref only (<code>me</code> , <code>here</code> , <code>#N</code> , <code>\$name</code>).	When you only want references, not names.

Note that `bmatch` returns one object, not a list — the difference from `pmatch` is dbref support, not arity. `match_all` is the one that returns a list.

Name matching understands how players actually talk:

- **Articles** are stripped (`the`, `a`, `an`, `some`).
- **Ordinals** select among duplicates: `2nd sword`, `third rope`, or a bare `3`.
- **Adjectives** filter in order: in `blue sword`, `sword` is the noun and `blue` must appear (as an in-order substring of the name, or in the object's `adjectives` list).
- **Aliases** and the atomic `noun` property are both checked, with multi-character tokens matched by prefix.

A standard player-verb matching block:

```
target = pmatch(dobj, pobj, list(pobj.location.contents))
if not target or not getattr(target, 'existent', False):
    pobj.msg("You don't see that here.")
    return
```

The `existent` check after matching is a project convention for **unhidden player verbs**: confirm the matched object still exists and is real before acting on it. Staff verbs on #3 use `bmatch` and generally skip that check.

Verb types: customizing the parser

The parsed command parts your verb reads (`dobj`, `prep`, `iobj`, `switches`, `lhs / rhs`, ...) don't appear by magic — they're produced by the verb's **verb type**, a small, swappable class that owns

the *parse* step and nothing else. Every verb has one, named by its `parent_type` : a dotted-path string stored in the `parent_type` column of the `verbs` table, defaulting to `moo.verb_types.MasterVerb` . The classes live in `moo/verb_types.py` .

Verb types are **engine Python**, not verb code. Changing one is a server-source change, not something you can do from `@program` — see [Adding a custom type](#) below. In day-to-day work you will use one of the two that ship, and most likely never think about this at all.

How a verb runs

Before your verb’s code executes, `_instantiate_verb_type()` (`moo/verb_namespace.py`) resolves the verb’s `parent_type` to a class via `resolve_verb_type()` , instantiates it, sets the runtime context on the instance (`pobj` , `this` , `location` , `db` , `cmdstring` , `raw` , `args` , and any injected switches), and calls:

```
inst.parse()
```

The attributes `parse()` leaves on the instance are then copied into [the namespace](#) your code sees. If the class can’t be instantiated or `parse()` raises, the engine logs a warning and falls back to simple string-split defaults, so the verb still runs with sensible `dobj` / `prep` / `iobj` values.

The lifecycle

Three of the four methods on `BaseVerb` are wired to the engine, and they fire around **every** execution of the verb — player commands, `call_verb` chains, and engine-invoked hooks like `go_` alike:

<code>at_pre_cmd()</code>	setup; return True to veto the command
<code>parse()</code>	split <code>self.args</code> into the standard slots
<the .py body>	your verb file — skipped if <code>at_pre_cmd</code> vetoed
<code>at_post_cmd()</code>	cleanup; runs even if the body raised or was vetoed

- **`at_pre_cmd()` runs before `parse()`** , which is what makes it the place for a check that should abort without paying for parsing — and equally why `dobj` / `prep` / `iobj` are *not* available to it yet. Return `True` to skip the body, the same “return True suppresses the default” convention the hook system uses.
- **`at_post_cmd()` always runs.** It reads the outcome off `self` : `self.result` (the body’s return value), `self.error` (the exception, or `None`), and `self.vetoed` .
- **Both fail safe.** An exception in `at_pre_cmd()` is logged and the command proceeds — one broken hook on a shared type must not silently swallow every command using it. An exception in `at_post_cmd()` is logged and swallowed, so cleanup can’t replace the failure it was reacting to.

func() is still not called by anything. Your verb's `.py` file *is* the body; there is no `func()` to override. It survives on `BaseVerb` only so that existing subclasses keep importing.

Example: one roundtime check instead of thirty

Every IC action verb opens with the same four lines — if the character is still in roundtime, refuse. A verb type turns that into a property of the verb rather than a paragraph each one repeats:

```
# in moo/verb_types.py (engine source)
@register_verb_type
class TimedVerb(MasterVerb):
    """An action a character in roundtime cannot take."""

    def at_pre_cmd(self):
        rt = getattr(self.pobj, 'rt', 0) or 0
        if rt > 0:
            self.pobj.msg(f"You must wait {rt} more seconds.")
            return True          # veto — the verb body never runs
```

Attach it and the check is inherited, with nothing left in the verb body:

```
add_verb(weapon, ['swing'], parent_type='moo.verb_types.TimedVerb')
```

Three things make this work where a plain early `return` in each verb would not:

- **The body never starts.** A veto is not an early return inside your code; the `.py` file is not executed at all, so there is no path through it that can forget the check.
- **It covers `call_verb` too.** A combat verb reached from another verb gets the same gate, which a copy-pasted guard at the top of player-facing verbs misses.
- **or 0 is load-bearing.** A missing property reads back as the `_null_attr` sentinel, which is falsy but is *not* `None` — `rt is None` is always `False`, and comparing the sentinel with `> 0` raises.

Because `at_pre_cmd()` runs before `parse()`, this pattern suits checks about the *actor* — roundtime, position, stun, permissions. A check that needs to know *what was targeted* has no `dobj` yet and belongs in the body.

The two built-in types

```
BaseVerb      Minimal – parse() is a no-op; args is the raw string.
└─ MasterVerb Standard dobj / prep / iobj / switches parsing (the default).
```

- **MasterVerb** does classic LambdaMOO-style parsing: it pulls MUSH switches off the verb name, finds the first preposition, and splits the rest into `dobj` / `prep` / `iobj` (plus `lhs` / `rhs`, a second preposition, and the list forms). This is what almost every verb wants.
- **BaseVerb** is “bring your own parser”: its `parse()` does nothing, so every parsed slot arrives empty — including `arglist`, which is `[]` rather than the split arguments. Only `args` / `argstr` are populated (they come from the namespace builder, not the verb type), so `args.split()` is on you. Use it for verbs whose syntax doesn’t fit the dobj/prep/iobj mould.

Remember that even under `MasterVerb` the parsed slots are **strings** — see [Parsed command parts](#). A verb type splits text; it never resolves objects. Matching is always your verb’s job.

Adding a custom type

There are two knobs, in increasing order of effort.

1. Swap the preposition regex (`rexp`). `MasterVerb.parse()` uses the class’s `rexp` in place of the global `PREP_REGEX` when one is set — the lightest way to change which words or separators split `dobj` from `iobj`.

2. Subclass and override `parse()`. Take full control of how `args` is split.

There is one rule that is easy to get wrong: **`parse()` must write its results into the standard slot names, because the engine harvests a fixed list.** `_parse_verb_inst_into_namespace()` copies exactly these attributes off the instance and nothing else:

```
dobj dobjlist prep preplist iobj iobjlist
dobj2 dobjlist2 prep2 lhs rhs arglist match switches
```

An attribute you invent — `self.field`, `self.value` — is **silently dropped**; your verb body will never see it. (`dobjstr` and `iobjstr` are derived from `dobj` / `iobj` at harvest time, so setting them directly does nothing either.)

So a `name: value` parser reuses the existing slots rather than inventing new ones:

```
# in moo/verb_types.py (engine source)
@register_verb_type
class KeyValueVerb(MasterVerb):
    """Parse 'name: value' pairs instead of dobj/prep/iobj."""
    def parse(self):
        key, sep, val = self.args.partition(':')
        self.dobj = key.strip()          # reads as dobj / dobjstr
        self.prep = ':' if sep else ''
        self.iobj = val.strip()         # reads as iobj / iobjstr
        self.lhs, self.rhs = self.dobj, self.iobj
        self.dobjlist = self.dobj.split()
        self.iobjlist = self.iobj.split()
        self.arglist = self.args.split()
        self.switches = getattr(self, '_injected_switches', []) or []
```

Anything you don't set keeps the empty default from `MasterVerb.__init__`, so a partial `parse()` degrades to blank slots rather than raising.

`@register_verb_type` is a **Python class decorator in engine source — not an in-game @-command**. It records the class in a registry keyed by its dotted path (`moo.verb_types.KeyValueVerb`) so `resolve_verb_type()` can find it by string. Registration is a fast path, not a requirement: `resolve_verb_type()` falls back to importing the module and doing a `getattr` for any path it doesn't already know.

Because this is engine code, the workflow is:

1. Add the class to `moo/verb_types.py` (or another module under `moo/` that the server process can import).
2. **Restart the server** (`@restart`). The auto-reload watcher only covers `moo verbs/`; engine modules are not hot-reloaded.
3. Attach it to a verb by setting that verb's `parent_type`.

Three ways to set `parent_type`:

How	Reaches
<code>@adverb <obj>.<name></code> with <code><perms></code> base	<code>BaseVerb</code> only — the in-game toolkit has no syntax for an arbitrary type.
<code>add_verb(obj, names,</code> <code>parent_type='moo.verb_types.KeyValueVerb')</code> from verb code	Any importable type. This is the normal route.
The JSON API <code>set_verb</code> request with a <code>parent_type</code> field	Any importable type. Note the MCP <code>set_verb</code> tool does not expose it — it passes only <code>objnum / verb / code</code> .

What the verb type does *not* control

`moo/verb_types.py` also defines `define_verb()`, `BaseVerb.matches()`, `to_dict()` / `from_dict()`, and the class attributes `key`, `aliases`, `min`, `perms`, and `help_text`. **None of these are used by the running engine** — nothing calls `define_verb()`, and verb identity lives entirely on the `VerbDef` (`moo/verbs.py`), which has its own `names`, `min_lengths`, `matches()`, `perms`, `hidden`, and `auth`. Setting them on a custom verb type has no effect; set them with `@adverb` / `@min` / `@verbauth` instead.

Messaging and emit substitution

There are three ways to put text in front of players, all of which run the text through the substitution engine (`moo/string_utils.py`) and then the recipient's color/wrap/screenreader pipeline:

- `pobj.msg(...)` — to one player.
- `room.msg_room(..., exclude=[pobj], sub=pobj, dob=target)` — to everyone in a room, optionally excluding actors.
- `broadcast(...)` — to everyone connected.

`msg` and `msg_room` are themselves verbs defined on #1 (`Root_Object`), so every object inherits them and verb code always reaches for `obj.msg(...)` rather than a free function. (Under the hood `msg` wraps a lower-level `notify()` primitive, but you write `obj.msg(...)`; because it's a verb, an object can even override it to filter or redirect its own messages — e.g. a deafened character.)

Substitution tokens

Pass the actors as `sub` (subject), `dob` (direct object), `iob` (indirect object), `uob` (noun source); the tokens then render per recipient:

Token	Renders
<code>%s</code> / <code>%S</code>	subject name / capitalized name
<code>%d</code> / <code>%D</code>	direct-object name / capitalized
<code>%i</code> / <code>%I</code>	indirect-object name / capitalized
<code>%u</code> / <code>%U</code>	noun (from <code>uob</code>) / capitalized
<code>%ps</code> <code>%po</code> <code>%pp</code> <code>%pa</code> <code>%pr</code>	gender pronouns of the subject: subjective / objective / possessive / possessive-adjective / reflexive

Both `%` and `$` prefixes are accepted for these tokens. Capitalize the token to capitalize the output (`%Ps` → “He”). Gender comes from the actor's `gender` property; the four supported

genders (male , female , neutral , ambiguous) map to pronoun sets in `moo/globals.py` , with `ambiguous` using singular *they*.

A canonical three-audience emit, from `moo verbs/17/tap.py` :

```
pobj.msg("You tap %d on the shoulder.", dob=tobj)
tobj.msg("%S taps you on the shoulder.", sub=pobj)
pobj.location.msg_room("%S taps %d on the shoulder.",
                       exclude=[pobj, tobj], sub=pobj, dob=tobj)
```

The actor sees “You tap Bob...”, Bob sees “Alice taps you...”, and the room sees “Alice taps Bob...” — one logical event, correctly conjugated for each viewer.

Pronoun helpers

For longer narrative strings, `su.psub1(text, eobj=...)` substitutes a single actor’s pronouns (`%N / %CN` name, `%EPS / %EP0 / %EPP / %EPR` and capitalized variants), and `su.psub2(text, eobj=..., tobj=...)` adds a target’s pronouns (`%T / %CT` , `%OPS / %OP0 / ...`). Reach for these whenever a verb narrates an interaction between two characters.

Calling other verbs

`call_verb(obj, verb_name, **kwargs)` runs another verb and returns its result. Extra keyword arguments are injected into the called verb’s namespace, which is the standard way to pass structured data between verbs:

```
# A player verb on the room dispatches to the object's implementation:
call_verb(exit, 'invoke')

# Pass structured data into the called verb's namespace:
call_verb(this, 'on_use', actor=pobj, target=dobj)
```

Switch syntax works here too: `call_verb(obj, 'look/brief')` . Nested calls are bounded by the maximum stack depth (50) to prevent runaway recursion.

This dispatch pattern is pervasive: room-level verbs are thin and call object-specific `_`-suffixed implementations. The room’s `sit` verb matches the furniture and calls `sit_` on it; `get / put / look` call the `in_ / on_ / under_ / behind_` family on the target object. Keeping the player-facing verb a dispatcher lets each object type customize behavior without re-implementing parsing.

Creating and changing objects

Verbs can build the world at runtime:

```
seat = create(parent=db.get_object(24))    # child of #24 (chair)
sword.noun = "sword"
sword.description = "A plain iron sword."
add_property(sword, 'damage', 12)
move(sword, pobj)                          # into the player's inventory
```

- `create(parent, owner)` returns a new `M000object`.
- Property writes are just attribute assignment (`sword.damage = 12`), or `add_property` when you want explicit perms.
- `move(obj, dest)` relocates an object and fires the before/after move hooks.
- `recycle(obj)` deletes it (sending it to the trash).
- `chparent(obj, new_parent)` reparents.

The higher-level `ou` module (`ou.make_room`, `ou.make_object`, ...) wraps these for common world-building shapes, and the `@`-command toolkit in Building Worlds is built on exactly these primitives.

Timing: pause, delay, and fork

Three builtins schedule work. The distinction that matters is **blocking vs. not**: verb code runs on a single shared worker thread, so anything that blocks stops the whole world.

Builtin	Blocking?	Use for
<code>pause(seconds)</code>	Yes — sleeps the shared worker	Short beats in a sequence you accept freezing for. Capped at 30s.
<code>delay(seconds, code, context)</code>	No	The normal way to schedule later work.
<code>fork(seconds, code, context)</code>	No	Independent follow-up that runs as its own task.

`delay` and `fork` take the deferred work as a **code string** plus a `context` dict of the names it should see — they do not take a callable:

```
pobj.msg("You begin picking the lock...")
delay(3, 'pobj.msg("...click. The lock opens.")', {'player': pobj, 'pobj': pobj})
```

`context` must contain `'player'`; `delay / fork` raise `ValueError` without it. Deferred code is preprocessed exactly like verb source (so `#N` and `$name` work) and re-runs on the same single worker.

There is **no** `suspend()` in the verb namespace. The task system (`moo/tasks.py`) has a `SUSPENDED` state that `delay / fork` drive internally, but verb code cannot park itself mid-execution and resume where it left off. To pause a *conversation*, use `yield`; to pause *work*, use `delay`.

Task limits. `TaskLimits` (`moo/tasks.py`) defines a tick budget (100,000), a wall-clock budget (5s), a verb-call stack depth of 50, and a fork depth of 10. These bound *queued tasks* — the ones `delay / fork` create. A top-level player command does not go through the task queue at all (see the request lifecycle); its guard is the 30-second `COMMAND_TIMEOUT`, and the stack-depth limit is enforced separately by `call_verb` via `MAX_VERB_DEPTH`.

Round time (the `rt` property) is the gameplay-level cooldown layered on top of all this: most action verbs early-return if `pobj.rt > 0` and call a helper to set round time after acting. That is a game convention, distinct from the scheduler's limits.

Interactive verbs with `yield`

A verb that needs to ask the player something mid-execution becomes a generator by using `yield`. Each `yield "prompt"` sends the prompt and resumes the verb with the player's next line:

```
ans = yield "This will overwrite the existing inscription. Proceed? (y/n) "
ans = (ans or '').strip().lower()
if ans[:1] == 'y':
    this.inscription = new_text
    pobj.msg("You carve the new inscription.")
else:
    pobj.msg("You leave it as it was.")
```

The engine detects the generator return value and drives an interactive session automatically. Multi-step new-character setup and builder confirmations both use this. No callback plumbing required — the verb reads like a linear conversation.

Permission-checked attribute access

The `getattr` / `setattr` injected into the namespace are **not** Python's builtins — they are wrappers (`moo/verb_namespace.py`) that enforce MOO property permissions when the target is a `M000bject`:

- **Read** is allowed if the property is readable (`r`), or the player owns it, or the player is a wizard.
- **Write** is allowed if the player owns the property and it is writable (`w`), or the player is a wizard.

Non-`M000bject` targets pass straight through. `hasattr` is the standard builtin and does *not* enforce permissions, which is why the idiom for “read a property that may be unset” is `getattr(obj, 'name', default)` rather than a `hasattr` guard:

```
rt = getattr(pobj, 'rt', None) or 0          # unset → 0
if getattr(exit, 'jumpable', False):        # unset → False
    ...
```

Conventions and idioms

These are project conventions (see also the codebase memory and existing verbs), not engine requirements, but following them keeps new content consistent:

- **Define in-game commands on the room parents — #16 (OOC) and #17 (IC) — not on player/character objects.** Characters carry only staff verbs (from #3). To give an individual object custom behavior, have the room verb call a `<verb>_` hook on the matched object rather than adding a player verb to the object.
- **Player verbs use `pmatch` ; staff verbs (on #3) use `bmatch` .**
- **Send player output with `obj.msg(...)` / `obj.msg_room(...)` , not a raw `notify()` — `msg` is the inherited verb (on #1) every object exposes.**
- **Unhidden player verbs check `obj.existent` after matching** and treat a non-existent object as no match.
- **Pass actors explicitly to messaging:** `sub=pobj, dob=dobj, iob=iobj` so `%S / %d / %i` resolve. For `gmove` on exits, pass `sub=player, dob=this` (the exit) so `$d` substitutes to the exit.
- **Round-time gate first.** Action verbs check `pobj.rt / position / status` before doing work and set round time after.
- **Dispatch to `_`-suffixed implementations.** Keep the typed verb a thin matcher/dispatcher; put behavior on the target object's `verb_` method.
- **Hidden / internal verbs** are named with a trailing `_` (called programmatically) or a leading `_` (ticker callbacks, lifecycle hooks) and are marked hidden so players can't invoke them.

- **Colors:** prefer `<245>` for dim UI chrome; avoid `<240>` (too dark); `&n` resets. All of it disappears under screenreader mode, so never encode meaning in color alone.
-

Next: Building Worlds — the in-game `@`-command toolkit that drives all of the above.